Reflective Synthesis Paper

GPU Infrastructure Scenario Reasoning Agent: An Integrative Industry Synthesis

Date: May 2026

Author: Sambasiva Andaluri

Executive Summary

Modern AI services run on expensive GPU hardware that often sits underutilized while operators struggle to make sense of alerts from multiple disconnected monitoring systems. When something goes wrong—a traffic surge, a hardware failure, or both at once—teams must manually piece together information from different dashboards to decide what to do.

This project creates an AI assistant that understands plain-English questions like "Traffic is spiking and we just lost two servers—what should we do?" The assistant combines insights from multiple specialized AI models: one that predicts how long requests will take, one that forecasts upcoming demand, and one that knows how to handle hardware problems. Instead of replacing human operators, it gives them a single, clear recommendation with an honest assessment of how confident the AI is in its advice.

The result is faster decision-making during critical incidents, with full transparency into how the AI reached its conclusions. This matters because GPU infrastructure powers everything from chatbots to image generators, and downtime or poor performance directly impacts customers and revenue.

---

1. Industry Context and Problem Definition

GPU cloud infrastructure has become the backbone of modern AI services, with demand for inference compute growing exponentially as large language models and generative AI applications proliferate. Organizations operating GPU clusters face a complex operational challenge: they must simultaneously optimize for efficiency (maximizing utilization of expensive hardware), reliability (minimizing service disruptions from hardware failures), and responsiveness (meeting latency SLAs despite variable demand patterns). Industry reports indicate that GPU utilization in production environments often averages below 50%, representing billions of dollars in underutilized capital expenditure (Stojkovic et al., 2025).

The operational complexity stems from the intersection of multiple concerns that traditional monitoring approaches handle in isolation. A traffic spike is a capacity problem; a GPU error is a reliability problem; a latency complaint is

an efficiency problem. But real incidents rarely fall neatly into one category—a hardware failure during a traffic spike creates a compound scenario where capacity planning, incident response, and latency prediction must work together. Current AIOps tooling typically addresses these concerns with separate dashboards, separate alerting pipelines, and separate runbooks, leaving operators to mentally synthesize fragmented information during high-pressure incidents.

This synthesis project addresses that gap by creating a scenario reasoning agent that integrates findings from four prior capstone projects into a unified analysis framework. The agent accepts natural-language what-if questions about GPU infrastructure scenarios and produces structured recommendations with transparent reasoning chains, explicitly connecting predictive analytics, demand forecasting, LLM-based advisory, and incident response workflows.

2. Overview of the Integrated Solution

The Scenario Reasoning Agent is an orchestration layer that routes infrastructure scenarios to appropriate analytical tools and synthesizes their findings. The agent follows a six-phase reasoning loop: UNDERSTAND (parse and classify the scenario), PLAN (select relevant tools), ANALYZE (invoke tools and gather evidence), SYNTHESIZE (combine findings into coherent analysis), RECOMMEND (produce prioritized actions with tradeoffs), and REFLECT (assess confidence and limitations).

The synthesis integrates four prior projects through three callable tools and an orchestration pattern:

- **Machine Learning Latency Predictor**: Applies the SHAP-identified feature relationships from the XGBoost latency prediction model—particularly the strong correlation (r=0.71) between model-specific rolling latency and request latency, and the impact of LoRA adapter loading overhead.
- **Deep Learning Demand Forecaster**: Applies the temporal pattern findings from the LSTM vs Transformer comparison—the strong 24-hour diurnal cycles, the Transformer's R^2=0.962 accuracy for 1-hour ahead forecasts, and the limited advance warning (~1 minute) for autoscaling decisions.
- **Generative AI Operations Advisor** (orchestration pattern, not a separate tool): Contributes the LLM advisory pattern for generating situation assessments and recommendations, along with the ethical warnings about over-reliance on fluent but potentially incorrect AI guidance. This project's approach is embedded in the synthesis agent's orchestration layer rather than exposed as a callable tool.
- **AIOps Incident Response System**: Applies the triage/diagnosis/remediation workflow with safeguards—severity classification, blast-radius checks,

peak-hours delays, and the cross-incident memory pattern for detecting recurrence.

The agent produces structured output including the full reasoning chain, tool invocation log, synthesized analysis, prioritized recommendations, confidence assessment, known limitations, and ethical considerations. This transparency requirement—making the reasoning process auditable—is a deliberate design choice reflecting the responsible AI principles identified in the Generative AI Operations Advisor and AIOps Incident Response System.

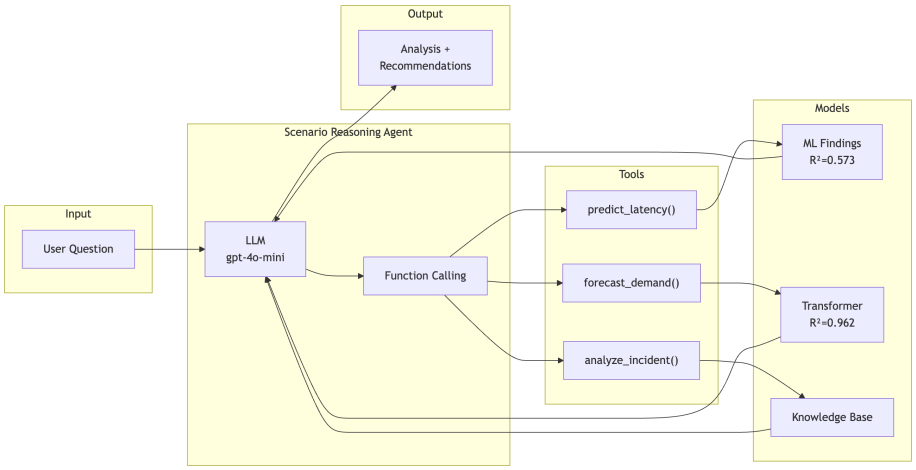

Figure 1: System Architecture

3. Integration of Prior Projects and Methods

The integration strategy follows a tool-composition pattern where each prior project contributes a specialized capability that the orchestration layer invokes as needed. This differs from a monolithic approach where all projects would be merged into a single model or pipeline.

Machine Learning Latency Predictor contributes predictive relationships between observable features and latency outcomes. The key insight is that model-specific historical performance is the strongest predictor—models that run slowly tend to continue running slowly. This finding enables the synthesis agent to reason about latency impact when scenarios mention specific models, adapter changes, or load increases. However, the model's R^2=0.573 means 43% of variance remains unexplained, which the synthesis explicitly acknowledges when assessing confidence.

Deep Learning Demand Forecaster contributes temporal structure understanding. The finding that demand patterns are largely linear and well-captured by 24-hour periodicity informs capacity planning advice: pre-scaling based on

time-of-day is often more effective than reactive autoscaling, given the ~1 minute advance warning limitation. The cross-service transfer failure (R^2=-1.743 when applying a code-service model to conversation traffic) also informs the synthesis—it warns that demand patterns are service-specific and should not be assumed to generalize.

Generative AI Operations Advisor contributes the advisory generation pattern and critical ethical awareness. The scenario augmentation's 342% error on NumImages distribution demonstrates that synthetic data can mislead capacity planning, which the synthesis cites when flagging limitations. The over-reliance warning—that LLM advisors produce fluent recommendations even when potentially wrong—is carried forward as a standard ethical consideration in all synthesis outputs.

AIOps Incident Response System contributes structured incident response patterns. The safeguard mechanisms (confidence thresholds, blast-radius checks, peak-hours delays) are directly applied when the synthesis handles reliability scenarios. The cross-incident memory pattern—escalating concern when the same error recurs on the same node—enables the synthesis to reason about degradation patterns rather than treating each event in isolation.

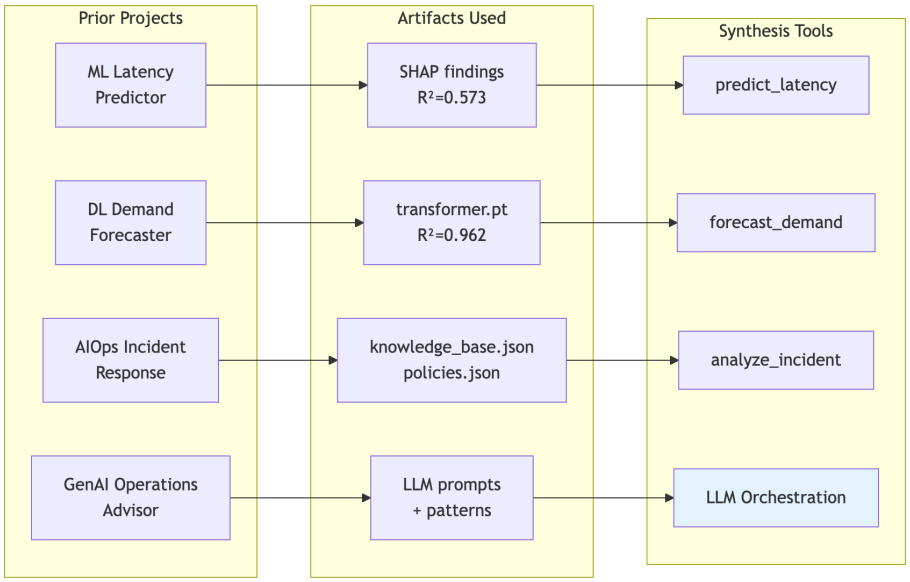

Figure 2: Model Integration

4. Technical Design Decisions and Tradeoffs

The choice to build an orchestration layer with tool-calling rather than a unified end-to-end model reflects several tradeoffs:

Modularity vs. Integration Depth: Keeping prior project capabilities as separate tools preserves their individual interpretability and allows independent validation. However, this means the synthesis cannot capture interaction effects that might emerge from joint training—for example, how latency predictions should be adjusted when demand forecasts indicate unusual traffic patterns.

LLM-Based Planning: The planning phase uses OpenAI function calling, where the LLM decides which tools are relevant based on tool descriptions rather than hardcoded rules. This provides flexibility for novel scenarios but introduces non-determinism—the same scenario might invoke different tool combinations on different runs. The tradeoff favors flexibility over determinism, appropriate for exploratory what-if analysis rather than production automation.

Mixed Model Loading Strategy: The Deep Learning Demand Forecaster loads and runs the actual trained PyTorch Transformer model, producing real numerical forecasts. The Machine Learning Latency Predictor uses an analytical model based on SHAP findings rather than loading the XGBoost pickle file (due to version compatibility constraints). This hybrid approach demonstrates both integration patterns: using trained model artifacts directly when possible, and encoding model findings analytically when artifact loading is impractical.

Transparency vs. Conciseness: The full reasoning chain output is verbose, which may overwhelm operators during time-critical incidents. A production version would need configurable verbosity levels—detailed chains for post-incident review, concise summaries for real-time response.

The synthesis differs from deterministic infrastructure simulators like Vidur (Agrawal et al., 2024) and llm-d-inference-sim in that it reasons about scenarios in natural language rather than simulating specific configurations. This makes it more flexible for what-if analysis but less precise for quantitative capacity planning.

5. Ethical, Governance, and Responsible AI Considerations

The synthesis inherits and amplifies ethical concerns from its component projects—the Machine Learning Latency Predictor, Deep Learning Demand Forecaster, Generative AI Operations Advisor, and AIOps Incident Response System. Three considerations are paramount:

Automation Bias: The Generative AI Operations Advisor identified that LLM advisors produce confident recommendations even when potentially incorrect. The synthesis compounds this risk by presenting integrated analysis from multiple sources—the appearance of triangulated evidence may increase operator trust beyond what the underlying confidence warrants. Mitigation: Every recommendation includes source attribution and explicit confidence assessment, and high-impact actions require human approval (following the AIOps Incident Response System's safeguard pattern).

Accountability Gap: When multiple AI components contribute to a recommendation, responsibility for errors becomes diffuse. If the synthesis recommends draining a node based on the incident response diagnosis, demand forecast, and advisory output, which component is accountable if the recommendation proves harmful? Mitigation: The reasoning chain provides complete auditability, allowing post-incident analysis to identify which component's contribution was problematic.

Fairness Across Workloads: The Generative AI Operations Advisor's finding that synthetic data over-represents multi-image workloads raises fairness concerns—capacity planning based on biased simulations may systematically under-serve text-only users (the majority). The synthesis must acknowledge this limitation rather than treating all tool outputs as equally reliable.

The synthesis follows the NIST AI Risk Management Framework's emphasis on transparency and human oversight (NIST, 2023). Every output includes explicit limitations, and the safeguard patterns from the AIOps Incident Response System ensure that recommendations remain advisory rather than automatically executed.

6. Limitations and Risks

The synthesis has several significant limitations:

Event-Driven Design: The tools only work when something is actively happening—a traffic spike, a hardware failure, or an alert firing. They cannot detect slow, gradual changes. For example, if GPU utilization quietly drops from 60% to 45% over a month with no alerts, none of the tools can help because there is no "event" to analyze. Detecting such trends would require time-series anomaly detection, which none of the prior projects provide.

Single-Infrastructure Training Data: The Machine Learning Latency Predictor used Alibaba data, the Deep Learning Demand Forecaster and Generative AI Operations Advisor used Azure data, and the AIOps Incident Response System used simulated data. The synthesis cannot claim generalization to other cloud providers, hardware configurations, or workload types.

No Uncertainty Quantification: The confidence scores are heuristic aggregations of individual tool confidences, not statistically grounded prediction intervals. Operators should treat them as rough indicators, not precise probabilities.

Compound Scenario Interaction: When all three tools contribute to a compound scenario, the synthesis combines their findings through simple aggregation. It cannot model how the tools' recommendations might interact or conflict in ways requiring arbitration.

7. Professional and Industry Relevance

This synthesis directly applies to my role at Oracle Cloud Infrastructure (OCI) managing GPU clusters for AI workloads. The operational challenges addressed—predicting latency impact of configuration changes, forecasting demand for capacity planning, generating actionable advisories, and structuring incident response—are daily concerns for infrastructure teams.

The skills demonstrated include system design (composing specialized components into unified workflows), ML integration (applying model findings as tools rather than running models), ethical reasoning (propagating responsible AI considerations through an integration layer), and cross-domain synthesis (connecting prediction, forecasting, generation, and agentic response into coherent analysis).

The scenario reasoning pattern could extend to other infrastructure domains—network operations, storage systems, database clusters—where operators face similar challenges of integrating multiple analytical signals into unified decision support.

8. Future Extensions or Improvements

Three improvements would significantly enhance the synthesis:

1. **Trend Detection Tool**: Add a time-series anomaly detection capability that analyzes metrics over configurable windows to detect gradual degradation, addressing the identified failure case.
2. **Causal Correlation**: Integrate deployment and configuration change logs to help diagnose root causes—when utilization declined, what changed?
3. **Feedback Loop**: Implement outcome tracking so recommendations can be validated against actual results, enabling confidence calibration over time.

A production implementation would also require integration with live telemetry sources (Prometheus, Grafana, vendor APIs), proper authentication and access controls, and performance optimization for real-time incident response.

References

Agrawal, A., et al. (2024). Vidur: A large-scale simulation framework for LLM inference. *Proceedings of Machine Learning and Systems (MLSys '24).*

National Institute of Standards and Technology. (2023). *AI risk management framework (AI RMF 1.0).* U.S. Department of Commerce.

Schick, T., et al. (2023). Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761.*

Stojkovic, U., et al. (2025). DynamoLLM: Designing LLM inference clusters for performance and energy efficiency. *IEEE International Symposium on High-Performance Computer Architecture (HPCA '25)*.

Yao, S., et al. (2023). ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations (ICLR)*.

Zhang, J., et al. (2025). ModServe: Modality- and stage-aware resource disaggregation for scalable multimodal model serving. *ACM Symposium on Cloud Computing (SoCC '25)*.